Lightweight Pothole Detection for Vehicle-Based Road Monitoring

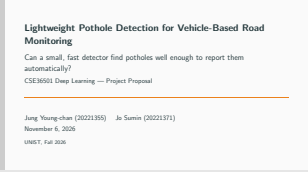
Can a small, fast detector find potholes well enough to report them automatically?

CSE36501 Deep Learning — Project Proposal

Jung Young-chan (20221355) Jo Sumin (20221371)

November 6, 2026

UNIST, Fall 2026

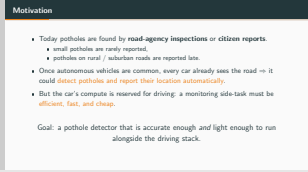


Hello, we are Jung Young-chan and Jo Sumin. Our project is lightweight pothole detection for vehicle-based road monitoring: can a detector be accurate enough, yet small and fast enough to run on a car? Today we cover the problem and the datasets.

Motivation

- Today potholes are found by **road-agency inspections** or **citizen reports**.
  - small potholes are rarely reported,
  - potholes on rural / suburban roads are reported late.
- Once autonomous vehicles are common, every car already sees the road ⇒ it could detect potholes and report their location automatically.
- But the car's compute is reserved for driving: a monitoring side-task must be efficient, fast, and cheap.

Goal: a pothole detector that is accurate enough *and* light enough to run alongside the driving stack.



Today, potholes are found by road-agency inspections or by citizen reports. But small potholes are rarely reported, and on rural roads reports come late. Once autonomous cars are common, every car is a road sensor: it could detect a pothole and send its location to the road agency right away. The catch is that the car's computer is busy driving, so a pothole detector, as a side task, must be efficient and fast.

Problem statement

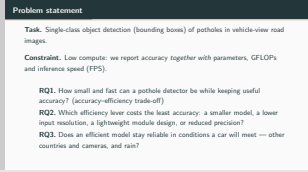
Task. Single-class object detection (bounding boxes) of potholes in vehicle-view road images.

Constraint. Low compute: we report accuracy *together with* parameters, GFLOPs and inference speed (FPS).

RQ1. How small and fast can a pothole detector be while keeping useful accuracy? (accuracy–efficiency trade-off)

RQ2. Which efficiency lever costs the least accuracy: a smaller model, a lower input resolution, a lightweight module design, or reduced precision?

RQ3. Does an efficient model stay reliable in conditions a car will meet — other countries and cameras, and rain?

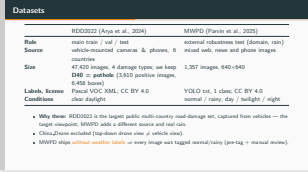


Formally, the task is single-class pothole detection with bounding boxes under a compute constraint, so every result is reported as accuracy together with model size, GFLOPs and speed. This gives three research questions. First, how small and fast can the detector get before accuracy becomes useless? Second, a model can be made cheaper with a smaller network, smaller input images, a lighter module, or lower numerical precision. Which one hurts accuracy least? Third, does the efficient model still work in other countries and in the rain?

Datasets

	RDD2022 (Arya et al., 2024)	MWPD (Parvin et al., 2025)
Role	main train / val / test	external robustness test (domain, rain)
Source	vehicle-mounted cameras & phones, 6 countries	mixed web, news and phone images
Size	47,420 images, 4 damage types; we keep **D40 = pothole** (3,610 positive images, 6,458 boxes)	1,357 images, 640×640
Labels, license	Pascal VOC XML; CC BY 4.0	YOLO txt, 1 class; CC BY 4.0
Conditions	clear daylight	normal / rainy; day / twilight / night

- **Why these:** RDD2022 is the largest public multi-country road-damage set, captured from vehicles — the target viewpoint; MWPD adds a different source and real rain.
- China_Drone excluded (top-down drone view $\neq$ vehicle view).
- MWPD ships without weather labels $\Rightarrow$ every image was tagged normal/rainy (pre-tag + manual review).



We use two datasets. RDD2022 is a large multi-country road damage dataset, about forty-seven thousand images captured from vehicles, which is exactly our viewpoint. We keep only the pothole class, D40, and drop the China drone subset because it is shot from above. MWPD, the Multi-Weather Pothole Detection dataset, is used only for testing: it comes from a different source and includes real rain. It has no weather labels, so we tagged every image ourselves.

What the data looks like



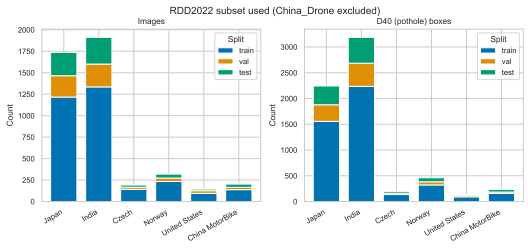
Random test images; green = pothole box (YOLO label).

- **RDD2022:** street scene from a car; potholes are small and far away.
- **MWPD normal:** web close-ups (watermarks); the pothole fills the frame.
- **MWPD rainy:** water-filled, reflective potholes.

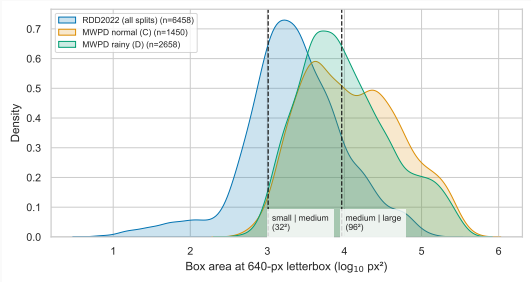


Here is what the data looks like. In RDD2022, on top, the camera sits in a car and sees the whole street, so potholes are small and far away. MWPD in normal weather, in the middle, is mostly close-up web photos where the pothole fills the frame. In the rain, at the bottom, potholes are filled with water and reflect the sky, so they look quite different.

Dataset analysis



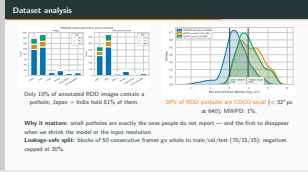
Only 10% of annotated RDD images contain a pothole; Japan + India hold 81% of them.



26% of RDD potholes are COCO-small ($< 32^2$ px at 640); MWPD: 1%.

Why it matters: small potholes are exactly the ones people do not report — and the first to disappear when we shrink the model or the input resolution.

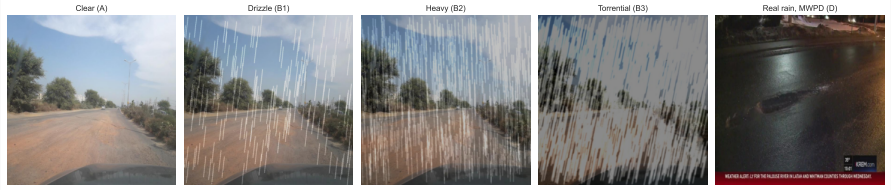
Leakage-safe split: blocks of 50 consecutive frames go whole to train/val/test (70/15/15); negatives capped at 20%.



Two observations. On the left, potholes are rare: only about one in ten annotated RDD images contains one, mostly from Japan and India. On the right, the key point: about a quarter of RDD potholes are small objects, under thirty-two pixels at the usual input size. Those are the potholes people never report, and the first a detector loses when we make it cheaper, so we report small-object accuracy separately. Finally, RDD has consecutive frames, so we split in blocks of fifty frames to keep near-identical images out of both train and test.

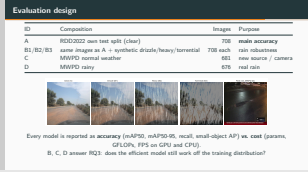
Evaluation design

ID	Composition	Images	Purpose
A	RDD2022 own test split (clear)	708	**main accuracy**
B1/B2/B3	*same images* as A + synthetic drizzle/heavy/torrential	708 each	rain robustness
C	MWPD normal weather	681	new source / camera
D	MWPD rainy	676	real rain



Every model is reported as **accuracy** (mAP50, mAP50-95, recall, small-object AP) **vs. cost** (params, GFLOPs, FPS on GPU and CPU).

B, C, D answer RQ3: does the efficient model still work off the training distribution?

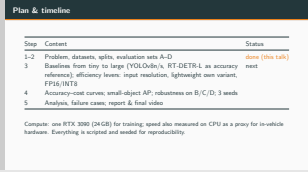


Set A, our clean RDD test split, measures the main accuracy. B1 to B3 are the same images with synthetic rain of increasing strength, and C and D are MWPD in normal weather and in real rain; together they answer the third question. Every model is reported on two axes: accuracy, including small objects, and cost, meaning parameters, GFLOPs and speed on a GPU and a CPU.

Plan & timeline

Step	Content	Status
1–2	Problem, datasets, splits, evaluation sets A–D	done (this talk)
3	Baselines from tiny to large (YOLOv8n/s, RT-DETR-L as accuracy reference); efficiency levers: input resolution, lightweight own variant, FP16/INT8	next
4	Accuracy–cost curves; small-object AP; robustness on B/C/D; 3 seeds	
5	Analysis, failure cases; report & final video	

Compute: one RTX 3090 (24 GB) for training; speed also measured on CPU as a proxy for in-vehicle hardware. Everything is scripted and seeded for reproducibility.



The problem and the data are done. Next we train baselines from tiny to large, apply the efficiency levers, and compare them with accuracy-versus-cost curves, including speed on a CPU as a stand-in for in-car hardware. Thank you.